

Migration: Monolithic to Microservices

MONOLITH TO MICROSERVICES MIGRATION GUIDE



Step 1: Project problems source identification

Ensure that the causes of your project issues aren't the following:

- insufficient business analysis
- poor team management
- unoptimized development processes
- insufficient team's seniority
- lack of testing



Step 2: Possible solutions analysis

Discover if project issues can be solved without converting a monolith to microservices.



Step 3: Monolith optimization

Optimize a monolith following the DDD/Clean Architecture guidelines to enable smooth refactoring.

1

¹DDD - Domain-driven design

Migration: Monolithic to Microservices cont'd



Step 4: Migration approach selection

Consider the best refactoring approach by choosing between a complete and incremental rewrite.



Step 5: Features prioritization

Analyze features that should be extracted and prioritize them by the following criteria.

- will accelerate development
- will solve project issues
- will help extract other services



Step 6: Features extraction

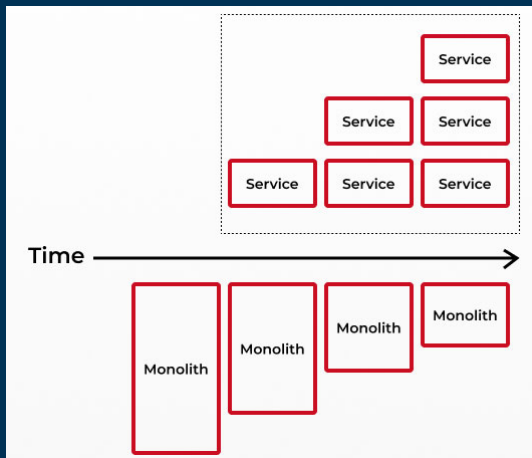
Create a strangler application by extracting the most important features and connecting them with a monolith.



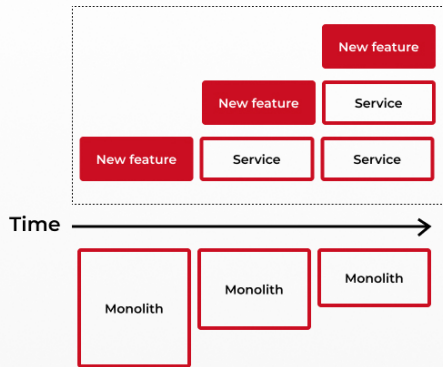
Step 7: New features implementation

Develop new features as distinctive microservices and connect them to a monolith.

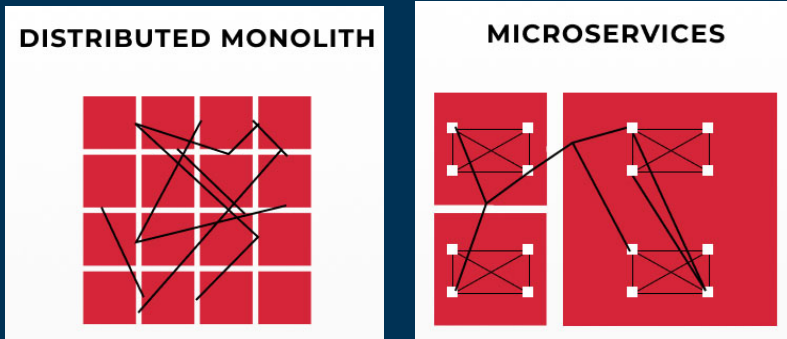
Migration: Monolithic to Microservices cont'd



NEW FEATURE IMPLEMENTATION



Migration: Monolithic to Microservices cont'd



- **Distributed monolith** describes an undesirable outcome of a monolith to microservices migration.
- It is characterized by extracted microservices that are highly connected or heavily dependent on a monolith.
- “Monolith issues at scale” remain unsolved, even though an app is split into microservices.